

Tuk-Tuk Tracking API - Executive Summary for Presentation

Quick Facts

- **Project Type:** RESTful Web API for Law Enforcement
- **Primary Language:** JavaScript (Node.js/Express)
- **Database:** MongoDB
- **Deployment:** Vercel (Live & Production)
- **Status:** ☒ Complete & Production-Ready
- **Student ID:** COBSCCOMP242P-055

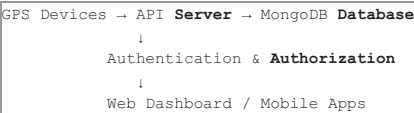
What Is This Application?

A real-time GPS tracking system designed for law enforcement agencies in Sri Lanka to monitor three-wheeler (tuk-tuk) vehicles in real-time. Think of it as a fleet management system with secure, role-based access for different levels of police administration.

Key Analogy: Similar to how delivery companies track delivery vehicles, this system tracks registered tuk-tuks with:

- Real-time GPS location updates
- Historical movement logs
- Secure role-based access
- Integration with Sri Lanka's province/district/station hierarchy

System Architecture in Plain English



The API acts as the central hub that:

1. Receives GPS location pings from devices
2. Stores and validates the data
3. Checks user permissions (role-based)
4. Returns information to authorized users

The 4 User Roles

1. CENTRAL ADMIN (HQ)
 - └ Full system control, manages everything
2. PROVINCIAL ADMIN
 - └ Manages 1 province, its districts & stations
3. STATION OFFICER
 - └ Views vehicles in their police station's area
4. DEVICE/DRIVER
 - └ Sends GPS location updates

What the System Tracks

Vehicles 🚲

- Registration number (with Sri Lankan format validation)
- Unique device ID (for GPS tracking)
- Current status (active/inactive/suspended)
- Vehicle details (make, model, color, year)

Drivers 👤

- Name, license number, contact info
- Age validation (must be 18-80)
- Emergency contact
- Vehicle assignment

Locations 📍

- Current position (latitude, longitude)
- Accuracy, speed, heading, altitude
- Timestamp for every location update
- Battery level and network type
- Complete history for investigations

Administrative Boundaries

- Provinces (9 in Sri Lanka)
- Districts (subdivisions of provinces)

- Police Stations (with coordinates)

The 30+ API Endpoints (Grouped)

Authentication (3 endpoints)

- Login
- Logout
- Refresh Token

Geography (11 endpoints)

- List/Get/Create/Update/Delete Provinces
- List/Get/Create/Update/Delete Districts
- List/Get/Create/Update/Delete Police Stations

Vehicle Management (6 endpoints)

- List/Get Vehicles
- Register/Update/Delete Vehicles
- Update vehicle status

Driver Management (5 endpoints)

- List/Get Drivers
- Register/Update/Delete Drivers

Location Tracking (5 endpoints)

- Submit GPS location ping
- Get latest locations for all vehicles
- Get location history for one vehicle
- Get current location of one vehicle
- Get location timeline

System (2 endpoints)

- Health check
- API status

Technology Stack - The Simple Version

Component	Technology	Why?
Server	Express.js (Node.js)	Fast, lightweight, perfect for APIs
Database	MongoDB	Flexible, great for real-time data, scales well
Authentication	JWT (JSON Web Tokens)	Secure, stateless, industry standard
Password Security	bcryptjs	Strong one-way hashing
Documentation	Swagger/OpenAPI	Interactive API testing & docs

Security Features

☒ JWT Token Authentication

- Every request needs a valid token
- Tokens expire (preventing old tokens from being used)
- Refresh tokens for getting new access tokens

☒ Role-Based Access Control

- Admin can do everything
- Provincial Admin limited to their province
- Station Officer limited to their station
- Device can only submit GPS data

☒ Password Security

- Passwords are hashed using bcryptjs
- Never stored as plain text
- Minimum 8 characters required

☒ Input Validation

- Registration numbers must match Sri Lankan format
- License numbers must match Sri Lankan format
- Email validation
- Age validation (18-80 for drivers)
- Coordinate validation (for locations)

☒ Account Protection

- Account lockout after failed login attempts
 - Track last login time
 - Activate/deactivate user accounts
 - Require password reset
-

Database Design

8 Collections (Tables) in MongoDB:

```
Provinces
├── Districts
│   └── Police Stations
│       └── Users (Station Officers)
└──
Vehicles
├── Drivers
└──
LocationPing (stores every GPS update)
├── Latest Location (most recent ping per vehicle)
└──
Users (all system users)
```

Example Data Flow:

```
GPS Device sends: {vehicleId: 123, lat: -6.92, lon: 80.77, speed: 45}
    ↓
LocationPing collection stores it
    ↓
LastKnownLocation collection updates (latest position)
    ↓
User queries "Show me vehicle 123's location"
    ↓
Response: Latest location data returned (fast lookup)
```

Real-World Use Case Example

Scenario: A stolen tuk-tuk is reported

1. **Dispatch Officer logs in** → Uses Station Officer credentials
2. **Searches for vehicle** → Returns location and movement history
3. **Sees real-time location** → Latest known position of vehicle
4. **Reviews history** → Sees where vehicle went in last 24 hours
5. **Sends location to patrol** → Provides coordinates to police
6. **Case solved** → Vehicle recovered, driver apprehended

Deployment Status

☑️ Already Live in Production

- **Base URL:** <https://tuk-tuk-tracking-api.vercel.app>
- **API Endpoint:** <https://tuk-tuk-tracking-api.vercel.app/api/v1>
- **API Docs:** <https://tuk-tuk-tracking-api.vercel.app/api/v1/docs>
- **Platform:** Vercel (serverless)
- **Backup:** Render (alternative platform)
- **Database:** MongoDB Atlas (cloud)

Key Strengths

Strength	Impact
Production-Ready	Can be deployed immediately with real data
Sri Lanka-Specific	Validates Sri Lankan vehicle registrations & licenses
Secure	Multi-layer security (JWT, RBAC, password hashing)
Scalable	Can handle hundreds of vehicles sending data simultaneously
Well-Documented	Clear API documentation with Swagger
Tested	Includes simulation tools and test data
Professional	Standardized responses, error handling, logging

What's Included in the Project

📁 Source Code (src/)

- 8 data models
- 7 controllers (business logic)
- 7 route modules (API endpoints)
- Authentication & authorization middleware
- Response utilities

📖 Documentation

- API_DESIGN.md (detailed specification)
- PROJECT_STRUCTURE.md (architecture)
- README.md (quick start)
- PROJECT_REPORT.md (comprehensive report)

🔧 Testing Tools

- Postman collection (for API testing)
- Database seeding script (populate test data)
- GPS simulator (simulate device location pings)

Configuration

- vercel.json (Vercel deployment)
- render.yaml (Render deployment)
- package.json (dependencies)

Quick Statistics

Metric	Value
Data Models	8
API Endpoints	30+
User Roles	4
Database Collections	8
Lines of Code	~2000+
Dependencies	9
Node Version Required	18+

Future Enhancement Ideas

Short Term

- Rate limiting (prevent API spam)
- Audit logging (track who did what)
- Advanced search filters
- Caching for better performance

Medium Term

- WebSocket for live location updates
- Movement analytics & reports
- Geofencing alerts
- Mobile app

Long Term

- Machine learning for anomaly detection
- SMS/Email notifications
- Integration with emergency services
- Advanced reporting dashboard

How to Present This

Slide 1: Introduction

- Project name and purpose
- One-sentence summary: "Real-time GPS tracking for law enforcement"

Slide 2: Problem Statement

- Law enforcement needs to track tuk-tuks
- Need secure access by different rank levels
- Need real-time and historical data

Slide 3: Solution Overview

- Shows system architecture diagram
- 4 user roles with access levels

Slide 4: Technical Stack

- Express.js + Node.js
- MongoDB
- JWT + RBAC security

Slide 5: Key Features

- Real-time GPS tracking
- Location history
- Role-based access
- Sri Lanka-specific validation

Slide 6: API Endpoints

- 30+ endpoints organized by resource
- Live API documentation

Slide 7: Security

- Authentication, authorization, encryption
- Input validation, account protection

Slide 8: Database Design

- 8 data models
- Relationships and indexes

Slide 9: Deployment

- Live on Vercel
- Alternative: Render
- MongoDB Atlas for database

Slide 10: Live Demo

- Show API documentation at: <https://tuk-tuk-tracking-api.vercel.app/api/v1/docs>
- Test an endpoint
- Show response

Slide 11: Strengths & Status

- Production-ready
- Well-documented
- Fully secure
- Ready for real-world use

Slide 12: Future Roadmap

- Short, medium, long-term enhancements

Slide 13: Q&A

- Open for questions

Live Demo Ideas

1. **Show API Documentation**
 - Navigate to: <https://tuk-tuk-tracking-api.vercel.app/api/v1/docs>
 - Show interactive Swagger documentation
 - Click "Try it out" on endpoints
2. **Submit a Location Ping**
 - Use `/locations/ping` endpoint
 - Show successful response
 - Explain what the data means
3. **Query Vehicles**
 - Use `/vehicles` endpoint
 - Show filtering by status
 - Demonstrate search functionality
4. **Check API Health**
 - Use `/health` endpoint
 - Show database connection status
 - Demonstrate system is operational

Key Takeaways

1. **Complete Solution:** Not just a simple API, but a full-featured tracking system
2. **Production-Ready:** Already deployed and operational
3. **Secure:** Multi-layer security appropriate for law enforcement
4. **Scalable:** Can handle growth and increased load
5. **Well-Built:** Professional code quality with proper architecture
6. **Documented:** Comprehensive docs for users and developers
7. **Real-World:** Solves actual problem for actual stakeholders

Files Reference

Document	Purpose
PROJECT_REPORT.md	Comprehensive technical report
API_DESIGN.md	Detailed API specification
README.md	Quick start guide
tuk-tuk-api-postman.json	Postman collection for testing
Live API	https://tuk-tuk-tracking-api.vercel.app/api/v1/docs

Questions to Prepare For

Q: Why MongoDB instead of PostgreSQL?

A: MongoDB is flexible for location data, great for real-time updates, and scales well horizontally for GPS data.

Q: How secure is the authentication?

A: Very secure - JWT tokens, bcryptjs password hashing, role-based access control, and input validation on every field.

Q: Can this scale to 1000s of vehicles?

A: Yes - MongoDB can handle millions of records, connection pooling supports concurrent requests, and Vercel auto-scales.

Q: What if database goes down?

A: The API returns error responses. In production, MongoDB Atlas provides replication and backups.

Q: How do GPS devices communicate?

A: Via the `/locations/ping` endpoint - they send location data via HTTPS POST request.

Version: 1.0.0

Date: May 28, 2026

Student ID: COBSCCOMP242P-055